

UNIT 1

JAVASCRIPT



Part 4 - Exercise

Client-side Web Development
2nd course – DAW
IES San Vicente 2026/2027
Arturo Bernal / Rosa Medina

Índice

Exercise JavaScript part 4.....	3
General considerations.....	6
Web services.....	7

Exercise JavaScript part 4

Download the included zip which contains the necessary files. It's a more complete version of the previous exercise . You can't modify any HTML file (only JS). Use **import/export** to use other functions, classes, etc from other files

constants.js

Here you'll define global constants for the app like the server's url.

http.class.js

This class contains the necessary HTTP methods for calling web services using GET, POST, PUT and DELETE. All methods return a promise with the response's data (or null if there's no data), or will throw an error if the server responds with an error.

categories.service.js

Create a class called **CategoriesService**. This class will have the necessary methods to get the list of product's categories from the server.

- Instantiate a Http class object inside a private attribute we'll use to make ajax calls
- **getAll** → Call `http://SERVER/categories` using GET. The server will return an object containing a property (categories) with the array:

```
{
  "categories": [
    {
      "id": 1,
      "name": "Informática"
    },
    {
      "id": 2,
      "name": "Telefonía"
    },
    ...
  ]
}
```

products.service.js

Create a class called ProductsService. This class will call to web services related to products (using the Http class).

- **getAll()** → Call `http://SERVER/products` using 'GET'. The server will return an object containing a property called products with the array:

```
{
  "products": [
    {
      "id": 8,
      "title": "Monitor Gaming 27 pulgadas 144Hz",
    }
  ]
}
```

```

        "description": "Monitor IPS 27\" 144Hz 1ms, resolución 1080p,
soporte regulable en altura, puertos HDMI y DisplayPort.",
        "price": 130,
        "status": 1,
        "datePublished": "2026-09-25T17:49:48.052Z",
        "numVisits": 31,
        "category": {
            "id": 1,
            "name": "Informática"
        },
        "mainPhoto": "http://localhost:3000/img/products/product-8.jpg",
        "photos": [
            {
                "id": 8,
                "url": "http://localhost:3000/img/products/product-8.jpg"
            }
        ]
    },
    ...
]
}

```

- **post(product)** → Call *http://SERVER/products* using 'POST', and send the product object. Example of a correct product to send:

```

{
  "title": "Test product",
  "description": "Test description",
  "price": 10,
  "category": 1,
  "mainPhoto": "Image in base 64"
}

```

The server will return a JSON object with the inserted product (which will contain the correct id and photo's url).

```

{
  "product": {
    "id": 13,
    ...
  }
}

```

- **delete(id)** → Will call *http://SERVER/products/:id* using 'DELETE' (:id is the id of the product). The server will return an empty response (204) if everything was ok, or an error if something went wrong.

new-product.js

This script will validate the form in this page (same as exercise 2) and send it to the server (as a JSON object). Call the corresponding method of the **ProductsService** class.

First of all, you'll have to load the categories and add them to the **select#category** element. The value of each option will be the id of that category.

Important: Preserve the first option (value="") when adding data.

If the product was inserted (no error), redirect to **index.html** (use `location.assign`).

If there was an error with the response, show an alert message and don't go anywhere.

You don't need to reset the form anymore, because the page will be destroyed when you redirect to index.html.

index.js

Call the corresponding method from **ProductsService** to load the products (if you use **async/await**, do this in an independent function!, don't use await in the main block). Create a card for every product using the provided template and add it to the DOM (#products-container).

For every product card created, control the click event on the delete button inside it. It will call the delete method on **ProductsService** and if the server deleted the product successfully, remove the card from the DOM. Before deleting a product, ask the user using a confirm dialog.

On the **#search** element, each time the user types something (input event), a new array of products will be created by filtering the original one (by title and description). Clear the #products-container element and put the filtered products inside.

General considerations

- When working with promises, it's ok to use **then** or **async/await**.
- The property field for the photo (JSON) is called **mainPhoto**.
- The description field now requires 10 characters minimum
- Look carefully at the object structure you get from the server
- When using liveServer, you must add the extension (.js) to the imported files. In the future we'll use other method and won't need it.

Web services

To run the services in your local computer, the code is in this repository: <https://github.com/arturober/sanvipop-services-lite>. Instructions:

- Install Git if not installed.
- Install NodeJS if not installed (LTS version)
- Open VSCode with no opened folder. Select Clone Git repository. The url is: <https://github.com/arturober/inmosanvi-services-lite.git>
- Open the services folder with VSCode.
 - Execute **npm install** and **npm start**.
 - Services will run on <http://localhost:3000>

There's a Postman collection also available to test the web services before implementing using them in your application:

<https://www.postman.com/downloads/>